



university of
 groningen

Languages and Machines

L8: Turing machines

Jorge A. Pérez

Bernoulli Institute for Math, Computer Science, and AI
University of Groningen, Groningen, the Netherlands



Regular $\leftrightarrow$ Finite State Machines (FSMs)

Context-free $\leftrightarrow$ Pushdown Machines

Context-sensitive $\leftrightarrow$ Linearly-bounded Machines

Decidable $\leftrightarrow$ Always-terminating Turing Machines

Semi-decidable $\leftrightarrow$ Turing Machines

Languages and Machines, Up to Here



- Finite state machines (FSMs), in different flavors, fully characterize regular languages.



- Finite state machines (FSMs), in different flavors, fully characterize regular languages.
- There are, however, languages that are not regular. Example:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

Hence, FSMs cannot recognize languages such as L_1 .



- Finite state machines (FSMs), in different flavors, fully characterize regular languages.
- There are, however, languages that are not regular. Example:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

Hence, FSMs cannot recognize languages such as L_1 .

- Context-free languages include regular languages, but also languages such as L_1 . Pushdown machines, in different flavors, use a stack to fully characterize context-free languages (including L_1).



- Finite state machines (FSMs), in different flavors, fully characterize regular languages.
- There are, however, languages that are not regular. Example:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

Hence, FSMs cannot recognize languages such as L_1 .

- Context-free languages include regular languages, but also languages such as L_1 . Pushdown machines, in different flavors, use a stack to fully characterize context-free languages (including L_1).
- There are, however, languages that are not context-free. Example:

$$L_2 = \{a^n b^n c^n \mid n \geq 0\}$$



- Finite state machines (FSMs), in different flavors, fully characterize regular languages.
- There are, however, languages that are not regular. Example:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

Hence, FSMs cannot recognize languages such as L_1 .

- Context-free languages include regular languages, but also languages such as L_1 . Pushdown machines, in different flavors, use a stack to fully characterize context-free languages (including L_1).
- There are, however, languages that are not context-free. Example:

$$L_2 = \{a^n b^n c^n \mid n \geq 0\}$$

- What kind of machines do we need to recognize L_2 ?

Turing Machines (TMs)



Turing Machines (TMs)



- A Turing machine (TM) may access and modify any memory position, using a sequence of elementary operations
- No limitation on the space/time available for a computation
- A finite state machine equipped with a **tape**, divided into **squares**, which can be written on as a result of a transition
- The **head** of the machine can move to the right or to the left, allowing the TM to read and manipulate the input as desired

Turing Machines (TMs)



- A Turing machine (TM) may access and modify any memory position, using a sequence of elementary operations
- No limitation on the space/time available for a computation
- A finite state machine equipped with a **tape**, divided into **squares**, which can be written on as a result of a transition
- The **head** of the machine can move to the right or to the left, allowing the TM to read and manipulate the input as desired

In other words, a transition:

- ▶ changes the state
- ▶ writes a symbol on the square scanned by the head
- ▶ moves the head

Turing Machines (TMs)



A (simple) **Turing machine** M is a quintuple $(Q, \Sigma, \Gamma, \delta, q_0)$ where

- Q is a set of **states**
- $q_0 \in Q$ is the **start state**
- Γ is the **tape alphabet**, a set of symbols disjoint from Q .
Contains a **blank symbol** $\sqcup$, not in Σ
- $\Sigma \subseteq \Gamma \setminus \{\sqcup\}$ is the **input alphabet**
- The transition function δ is a partial function such that

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

If $\delta(q, X)$ is undefined then $\delta(q, X) = \perp$.

Turing Machines (TMs)



A (simple) **Turing machine** M is a quintuple $(Q, \Sigma, \Gamma, \delta, q_0)$ where

- Q is a set of **states**
- $q_0 \in Q$ is the **start state**
- Γ is the **tape alphabet**, a set of symbols disjoint from Q .
Contains a **blank symbol** $\sqcup$, not in Σ
- $\Sigma \subseteq \Gamma \setminus \{\sqcup\}$ is the **input alphabet**
- The transition function δ is a partial function such that

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

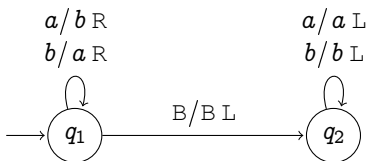
If $\delta(q, X)$ is undefined then $\delta(q, X) = \perp$.

Accepting states $F \subseteq Q$ not indispensable for defining acceptance (see later).

Example 1



A TM that reads the input string and interchanges symbols a and b :



A transition states: a *symbol before*, a *symbol after*, and a *direction*.

This way, for instance, in state q_1 , label ' $a/b \text{ R}$ ' indicates:

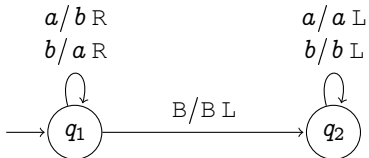
- symbol a is rewritten into b , and
- the head moves right (R).

We may abbreviate $a/a \text{ L}$ to $a/. \text{ L}$.

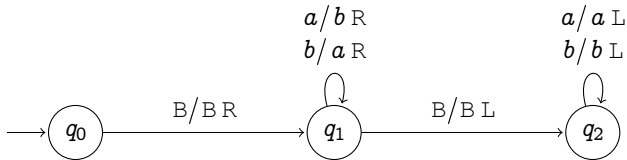
Example 1



A TM that reads the input string and interchanges symbols a and b :



A slightly more general machine:





The global state of the TM is determined by the state $q \in Q$, the contents of the tape (a string in Γ^*) and the position of the head

- A **configuration** of the TM is a string uqv in $\Gamma^* Q \Gamma^*$, in which:
 - u is a string on the tape to the left of the head
 - q is the **current** state
 - v is a string on the tape that begins under the head
- The **initial configuration** is $q_0 w$, where $w \in \Sigma^*$ is the input string
- The first symbol of v_{B^∞} is called the **current** symbol

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is $u Z Y r v$.

-

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is $u Z Y r v$.
 - Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u Z Y r v$.

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, \mathbb{R})$, then the next configuration from $u Z q X v$ is $u Z Y r v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u Z Y r v$.
2. If $\delta(q, X) = (r, Y, \mathbb{L})$, then the next configuration is



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is $u Z Y r v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u Z Y r v$.
2. If $\delta(q, X) = (r, Y, L)$, then the next configuration is $u r Z Y v$.

-

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is $u Z Y r v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u Z Y r v$.
2. If $\delta(q, X) = (r, Y, L)$, then the next configuration is $u r Z Y v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u r Z Y v$.
3. Otherwise, if $\delta(q, X) = \perp$, the next configuration is

Turing Machines (TMs)



Suppose that the **current configuration** is

$$u Z q X v$$

where X, Y, Z are tape symbols (in Γ). The head reads X .

To obtain the **next configuration**, we must (i) rewrite the symbol on the tape and (ii) move the head according to the direction.

Three options:

1. If $\delta(q, X) = (r, Y, R)$, then the next configuration from $u Z q X v$ is $u Z Y r v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u Z Y r v$.
2. If $\delta(q, X) = (r, Y, L)$, then the next configuration is $u r Z Y v$.
- Stepwise: $u Z q X v \rightarrow u Z q Y v \rightarrow u Z r Y v \rightarrow u r Z Y v$.
3. Otherwise, if $\delta(q, X) = \perp$, the next configuration is $\perp$.



More formally, we define a transition relation $\vdash$ on configurations:

$$\delta(q, X) = (r, Y, R) \Rightarrow u Z q X v \vdash u Z Y r v$$

$$\delta(q, X) = (r, Y, L) \Rightarrow u Z q X v \vdash u r Z Y v$$

$$\delta(q, X) = \perp \Rightarrow u Z q X v \vdash \perp$$



More formally, we define a transition relation $\vdash$ on configurations:

$$\delta(q, X) = (r, Y, R) \Rightarrow u Z q X v \vdash u Z Y r v$$

$$\delta(q, X) = (r, Y, L) \Rightarrow u Z q X v \vdash u r Z Y v$$

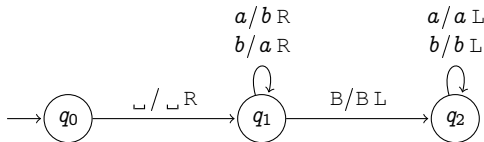
$$\delta(q, X) = \perp \Rightarrow u Z q X v \vdash \perp$$

- A computation is a sequence of steps, as defined by $\vdash$
- A TM **computes a function** f
 - if starting in $q_0 w, \dots$
 - ... the final tape upon termination is always $B^\infty u B^\infty$, with $u = f(w)$.

Example 1, Revisited



Consider a variation of the previous machine with $\Sigma = \{a, b, \sqcup\}$, where ' $\sqcup$ ' is always at the beginning of all input strings. We have:



Computation for input $abab$:

$\rightarrow [q_0] \sqcup a b a b B$
 $\vdash_{\sqcup} [q_1] a b a b B$
 $\vdash_{\sqcup} b [q_1] b a b B$
 $\vdash_{\sqcup} b a [q_1] a b B$
 $\vdash_{\sqcup} b a b [q_1] b B$
 $\vdash_{\sqcup} b a b a [q_1] B$

$\vdash_{\sqcup} b a b [q_2] a B$
 $\vdash_{\sqcup} b a [q_2] b a B$
 $\vdash_{\sqcup} b [q_2] a b a B$
 $\vdash_{\sqcup} [q_2] b a b a B$
 $\vdash [q_2] \sqcup b a b a B$

Example 2



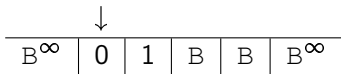
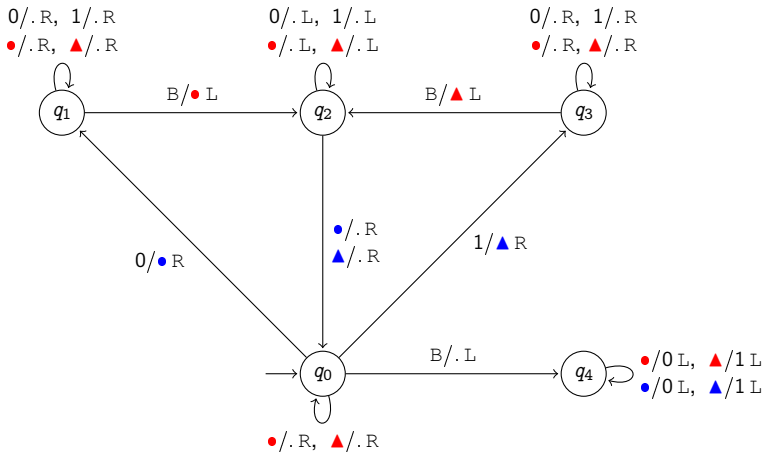
A TM that duplicates the input string $w \in \{0, 1\}^*$.

- ▶ Before: A tape with the string w
- ▶ After: The tape contains the string $w w$
- ▶ What is your (programming) strategy?

Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

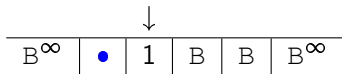
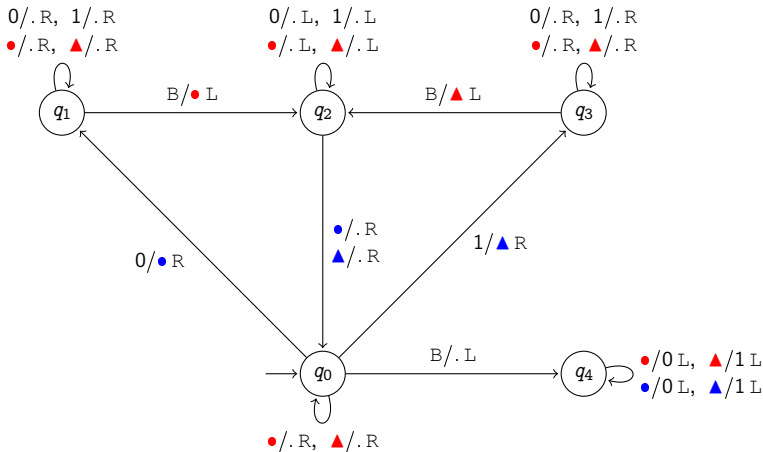


(State = q_0)

Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

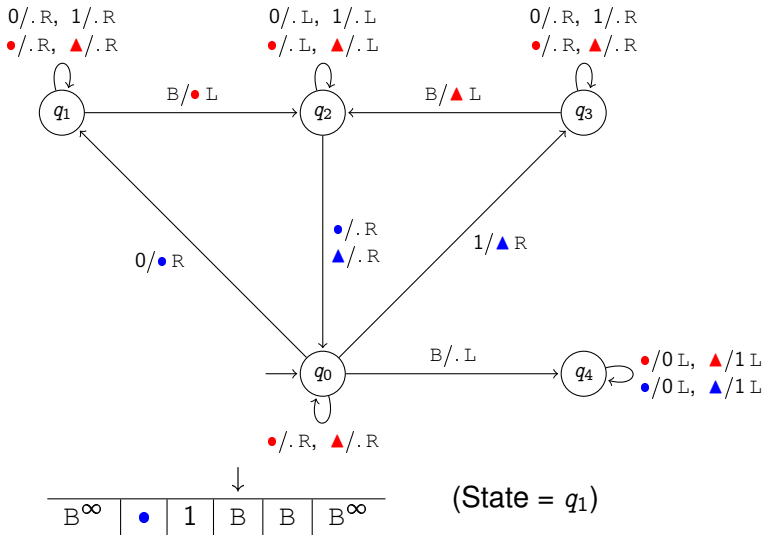


(State = q_1)

Example 2



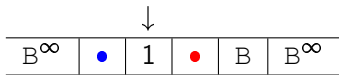
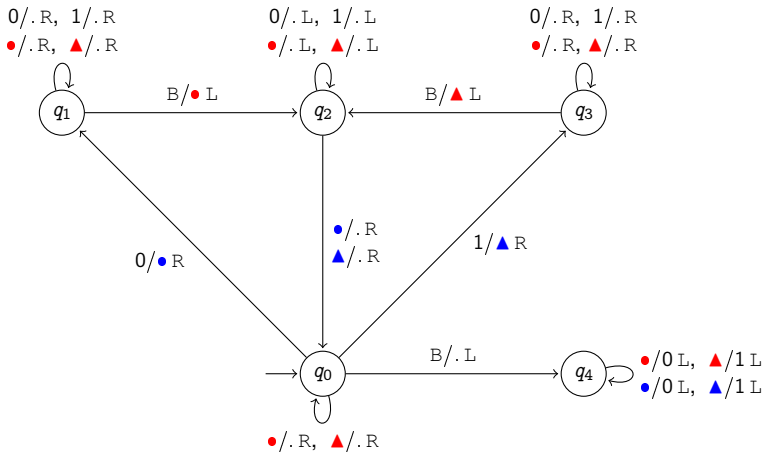
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

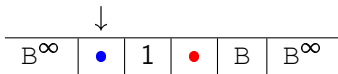
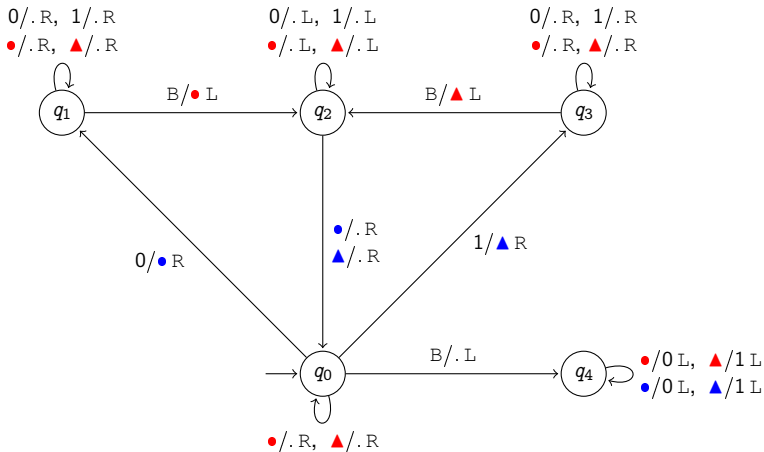


(State = q_2)

Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

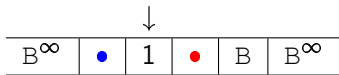
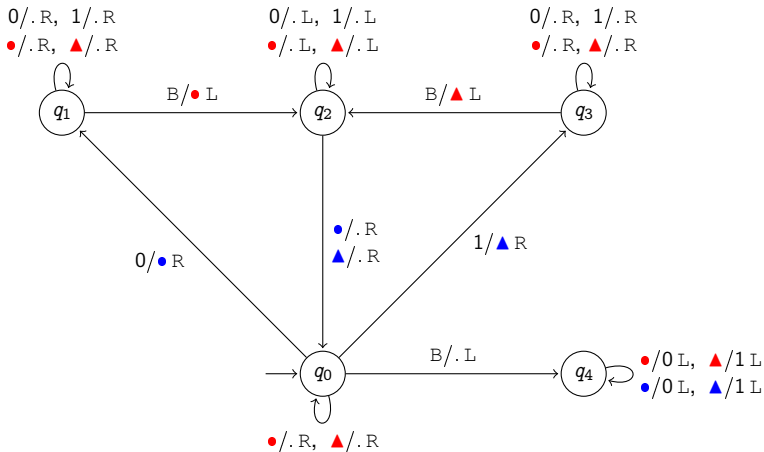


(State = q_2)

Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

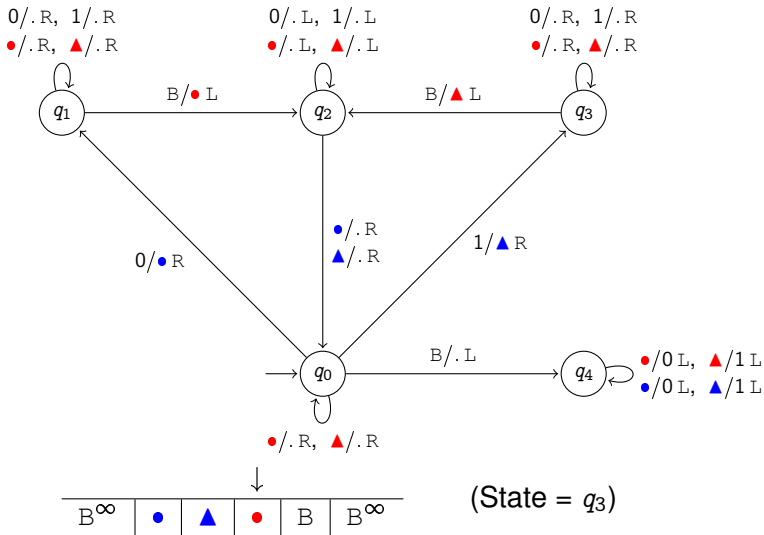


(State = q_0)

Example 2



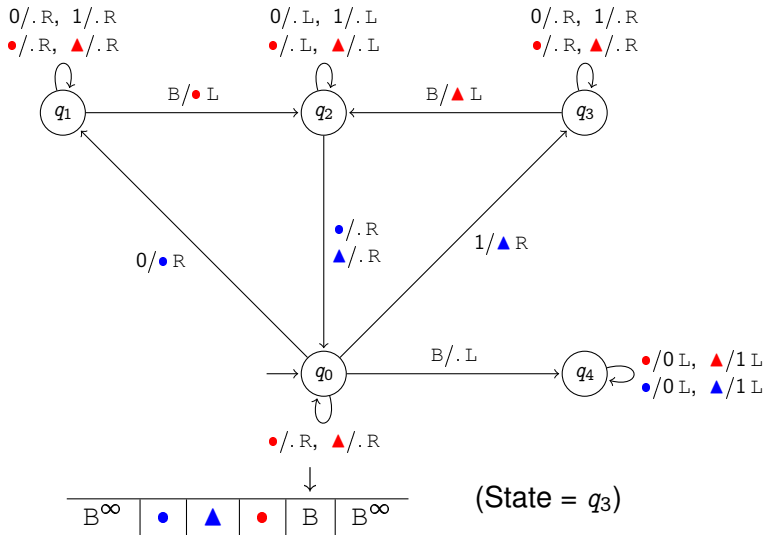
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



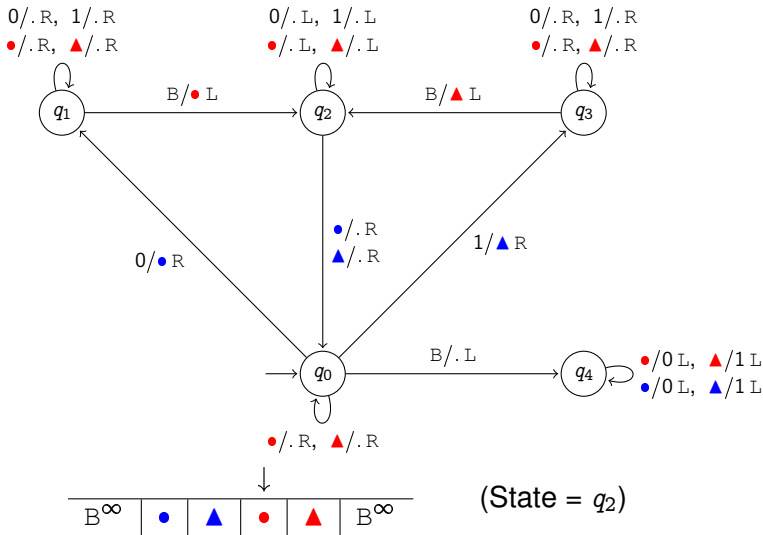
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



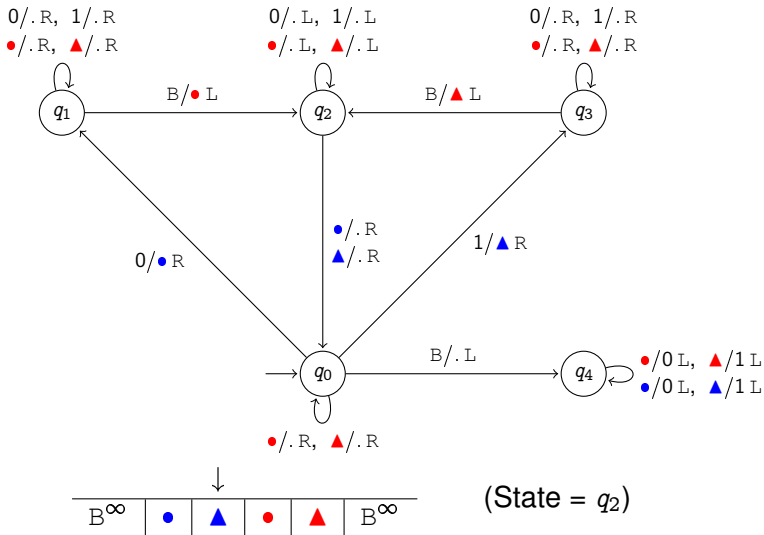
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



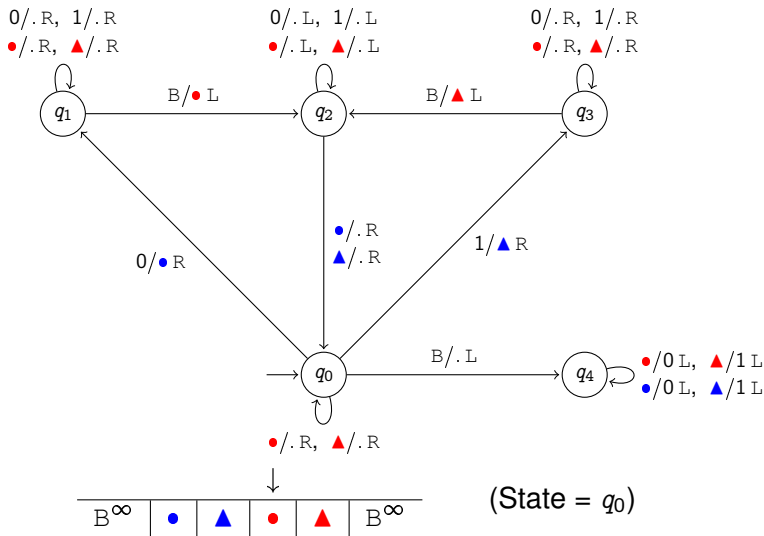
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



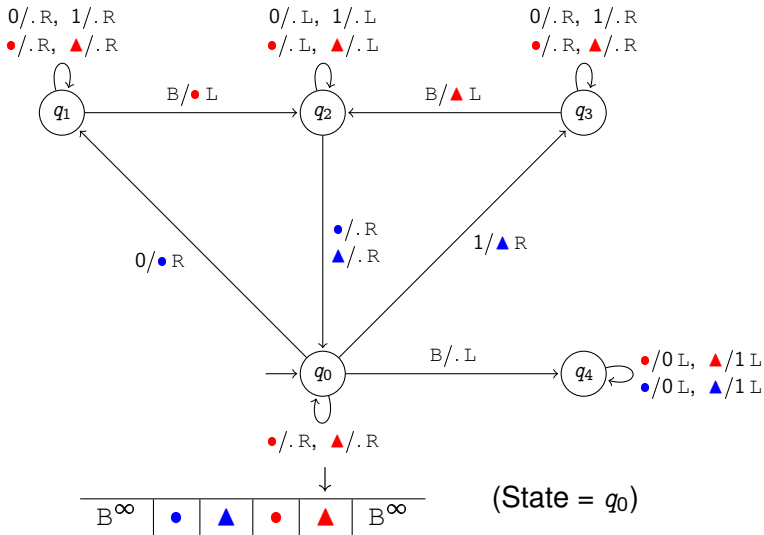
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



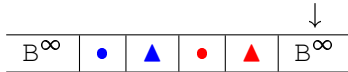
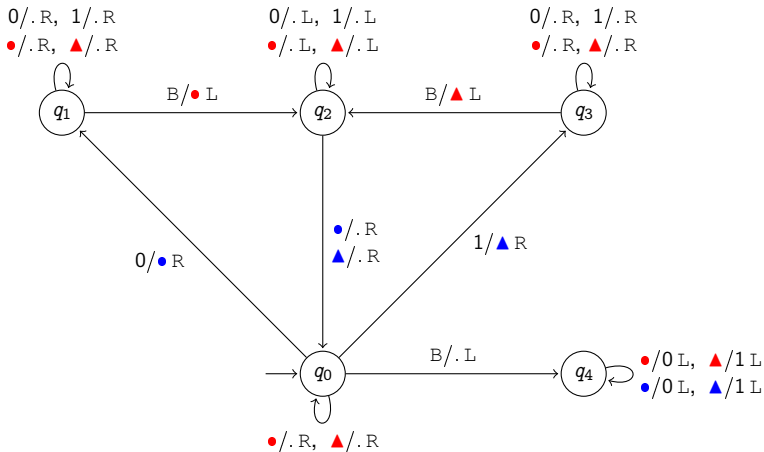
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

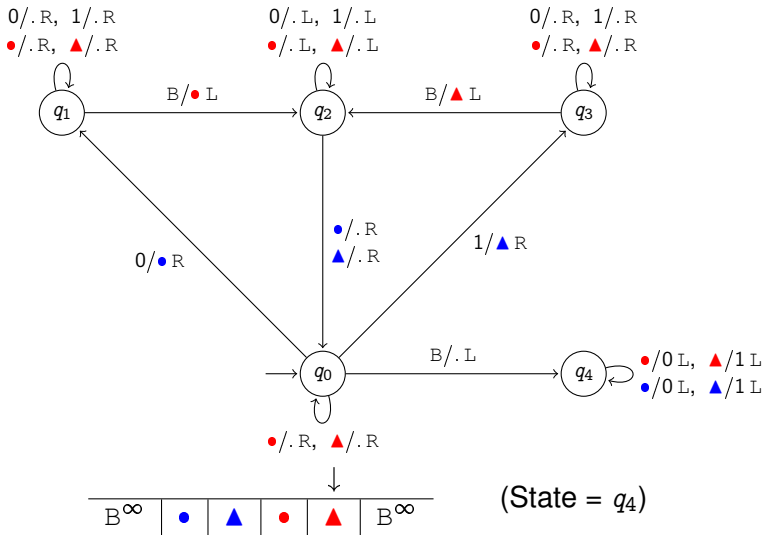


(State = q_0)

Example 2



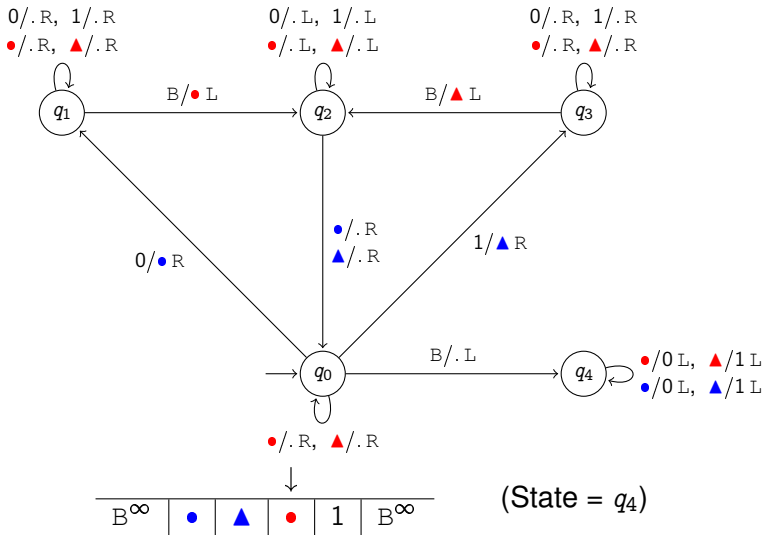
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



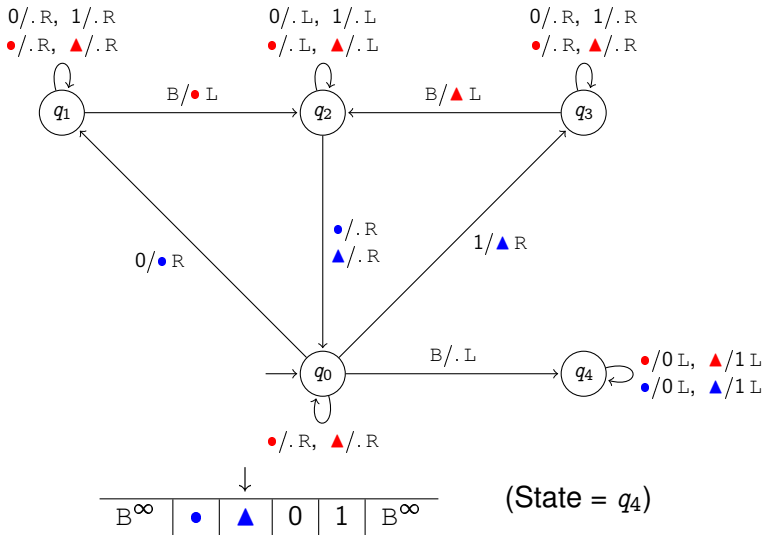
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



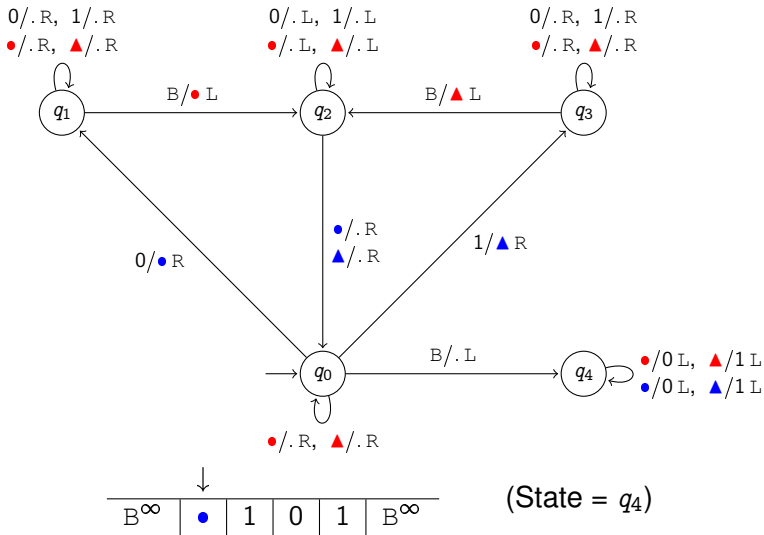
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



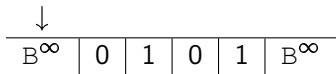
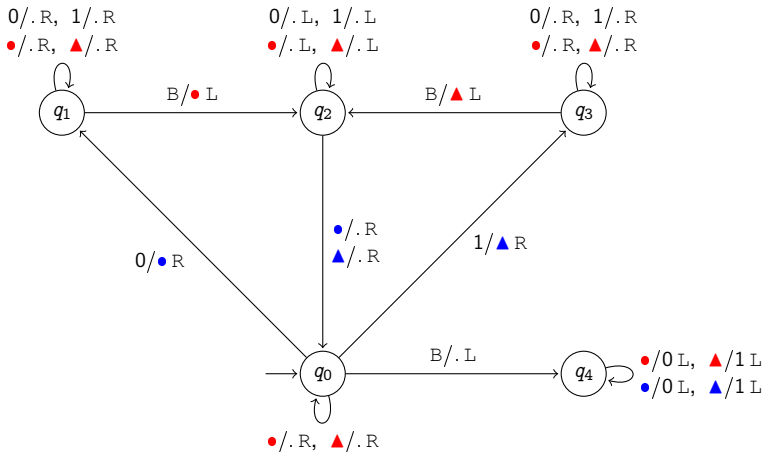
A TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

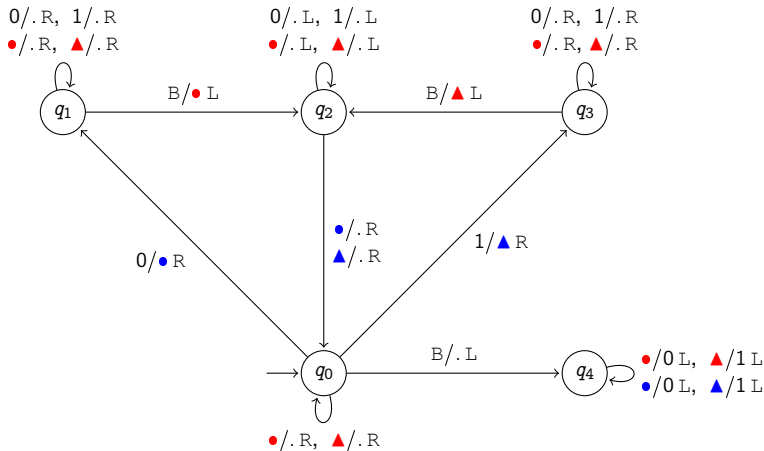


(State = q_4)

Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.



What does each state/transition represent?



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.

2. $L(M)$ can also be defined by **termination in an accepting state**, extending M with a set $F \subseteq Q$:

$$L(M) = \{w \in \Sigma^* \mid \exists q_f \in F, u, v \in \Gamma^* : q_0 w \vdash^* u q_f v \vdash \perp\}$$



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.

2. $L(M)$ can also be defined by **termination in an accepting state**, extending M with a set $F \subseteq Q$:

$$L(M) = \{w \in \Sigma^* \mid \exists q_f \in F, u, v \in \Gamma^* : q_0 w \vdash^* u q_f v \vdash \perp\}$$

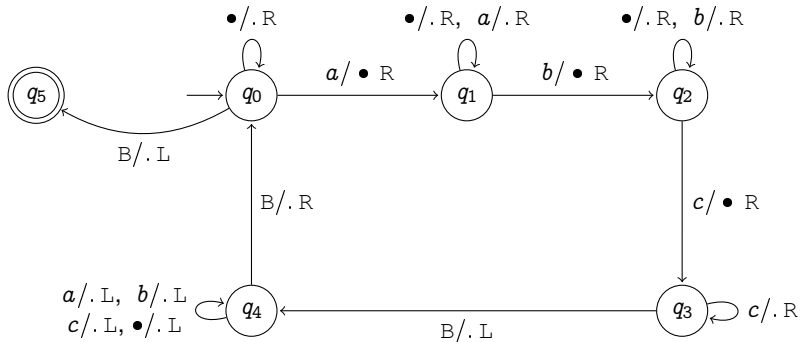
This definition can be reduced to the first one by letting $F = Q$.

In fact, both definitions are equivalent.

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$

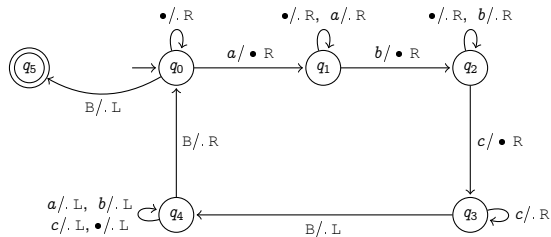


A TM with accepting state(s):



How does it work?

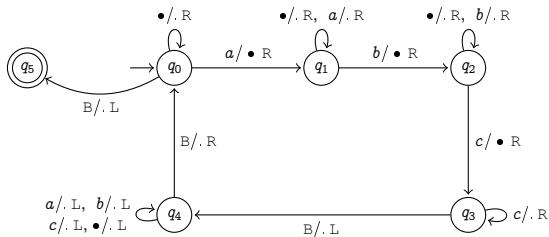
Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$

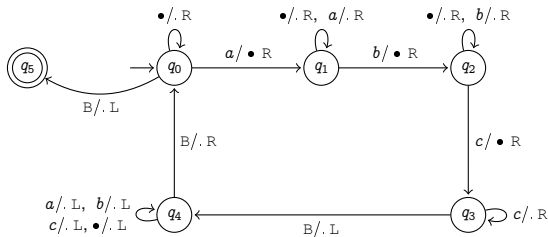


Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



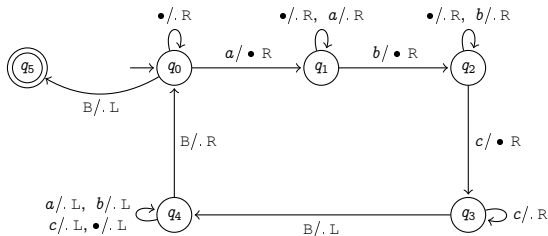
Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

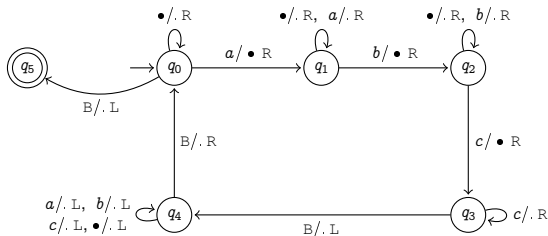
Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$
 $\vdash B \bullet [q_1] a b b c c B$
 $\vdash B \bullet a [q_1] b b c c B$
 $\vdash B \bullet a \bullet [q_2] b c c B$
 $\vdash B \bullet a \bullet b [q_2] c c B$

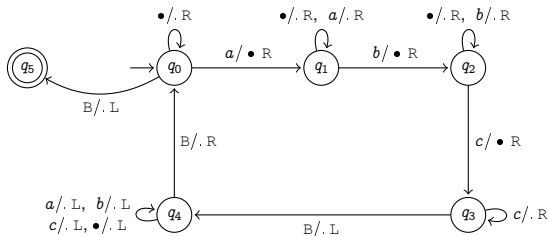
Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$
 $\vdash B \bullet [q_1] a b b c c B$
 $\vdash B \bullet a [q_1] b b c c B$
 $\vdash B \bullet a \bullet [q_2] b c c B$
 $\vdash B \bullet a \bullet b [q_2] c c B$
 $\vdash B \bullet a \bullet b \bullet [q_3] c B$

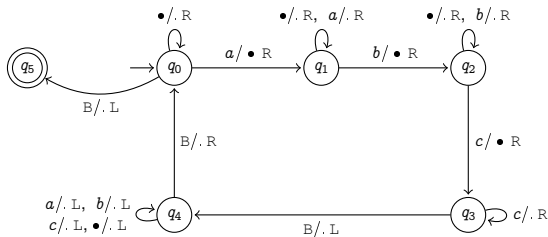
Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$
 $\vdash B \bullet [q_1] a b b c c B$
 $\vdash B \bullet a [q_1] b b c c B$
 $\vdash B \bullet a \bullet [q_2] b c c B$
 $\vdash B \bullet a \bullet b [q_2] c c B$
 $\vdash B \bullet a \bullet b \bullet [q_3] c B$
 $\vdash B \bullet a \bullet b \bullet c [q_3] B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

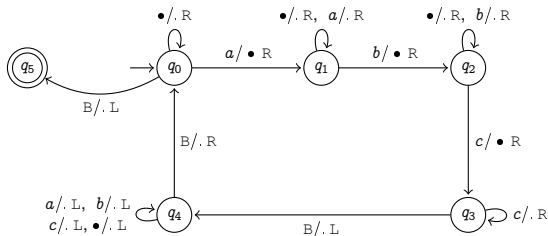
$\vdash B \bullet a \bullet b [q_2] c c B$

$\vdash B \bullet a \bullet b \bullet [q_3] c B$

$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

$\vdash B \bullet a \bullet b [q_2] c c B$

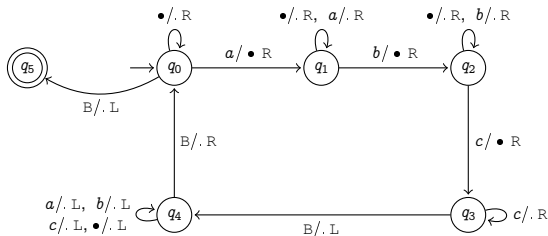
$\vdash B \bullet a \bullet b \bullet [q_3] c B$

$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

$\vdash^* [q_4] B \bullet a \bullet b \bullet c B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

$\vdash B \bullet a \bullet b [q_2] c c B$

$\vdash B \bullet a \bullet b \bullet [q_3] c B$

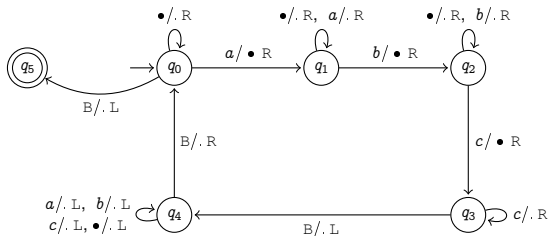
$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

$\vdash^* [q_4] B \bullet a \bullet b \bullet c B$

$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_3] B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$

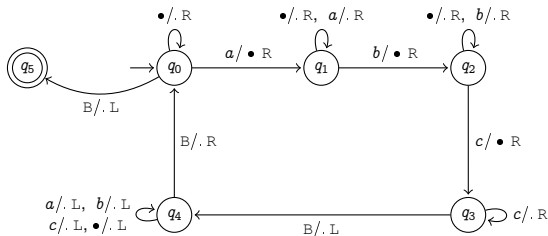


Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$
 $\vdash B \bullet [q_1] a b b c c B$
 $\vdash B \bullet a [q_1] b b c c B$
 $\vdash B \bullet a \bullet [q_2] b c c B$
 $\vdash B \bullet a \bullet b [q_2] c c B$
 $\vdash B \bullet a \bullet b \bullet [q_3] c B$
 $\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$
 $\vdash^* [q_4] B \bullet a \bullet b \bullet c B$
 $\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_3] B$
 $\vdash^* [q_4] B \bullet \bullet \bullet \bullet \bullet B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

$\vdash B \bullet a \bullet b [q_2] c c B$

$\vdash B \bullet a \bullet b \bullet [q_3] c B$

$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

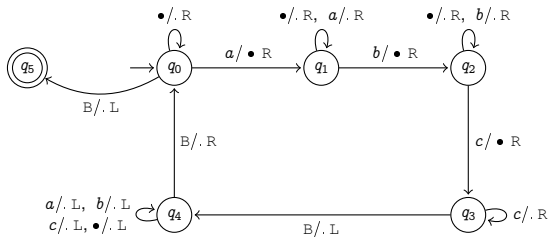
$\vdash^* [q_4] B \bullet a \bullet b \bullet c B$

$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_3] B$

$\vdash^* [q_4] B \bullet \bullet \bullet \bullet \bullet B$

$\vdash B [q_0] \bullet \bullet \bullet \bullet \bullet B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

$\vdash B \bullet a \bullet b [q_2] c c B$

$\vdash B \bullet a \bullet b \bullet [q_3] c B$

$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

$\vdash^* [q_4] B \bullet a \bullet b \bullet c B$

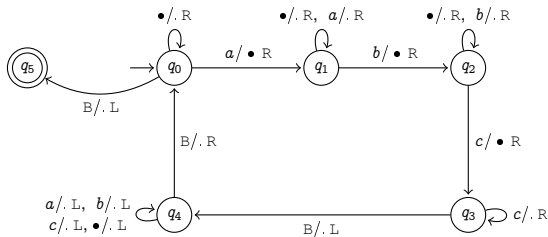
$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_3] B$

$\vdash^* [q_4] B \bullet \bullet \bullet \bullet \bullet B$

$\vdash B [q_0] \bullet \bullet \bullet \bullet \bullet B$

$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_0] B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Computation for input $aabbcc$:

$\rightarrow B [q_0] a a b b c c B$

$\vdash B \bullet [q_1] a b b c c B$

$\vdash B \bullet a [q_1] b b c c B$

$\vdash B \bullet a \bullet [q_2] b c c B$

$\vdash B \bullet a \bullet b [q_2] c c B$

$\vdash B \bullet a \bullet b \bullet [q_3] c B$

$\vdash B \bullet a \bullet b \bullet c [q_3] B$

$\vdash B \bullet a \bullet b \bullet [q_4] c B$

$\vdash^* [q_4] B \bullet a \bullet b \bullet c B$

$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_3] B$

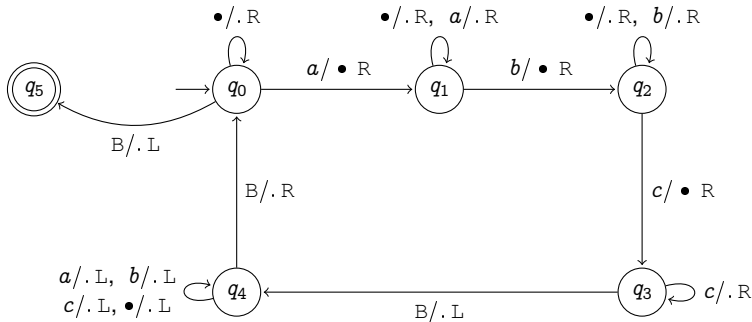
$\vdash^* [q_4] B \bullet \bullet \bullet \bullet \bullet B$

$\vdash B [q_0] \bullet \bullet \bullet \bullet \bullet B$

$\vdash^* B \bullet \bullet \bullet \bullet \bullet [q_0] B$

$\vdash B \bullet \bullet \bullet \bullet \bullet [q_5] \bullet B$

Example 5.2.1: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$



Consider now the computation for $aabcc$: where does it get stuck?



A TM is **always terminating** if it terminates for every input.

Let L be a language.

- L is **semi-decidable** (or **recursively enumerable, RE**)
if there exists a TM M such that $L = L(M)$.
- L is **decidable** (or **recursive**)
if there is an always terminating TM that accepts L by termination in an accepting state.
- If L is decidable, then it is also semi-decidable.
The converse doesn't hold!



This lecture (Sections 5.1 and 5.2):

- ▶ Turing machines
- ▶ Key terminology for TM-accepted languages

Next Lecture (Sections 5.3–5.8)

- Further examples of TMs
- Variants of TMs: multiple-track, multiple-tape, non-deterministic